UNIT-IV

State, State Graphs and Transition testing:- state graphs, good & bad state graphs, state testing, Testability tips.

Introduction

1. The finite state machine is as fundamental to software engineering as boolean algebra to logic.
2. State testing strategies are based on the use of finite state machine models for software structure, software behavior, or specifications of software behavior.
3. Finite state machines can also be implemented as table-driven software, in which case they are a powerful design option.

State Graphs

A state is defined as: "A combination of circumstances or attributes belonging for the time being to a person or thing."

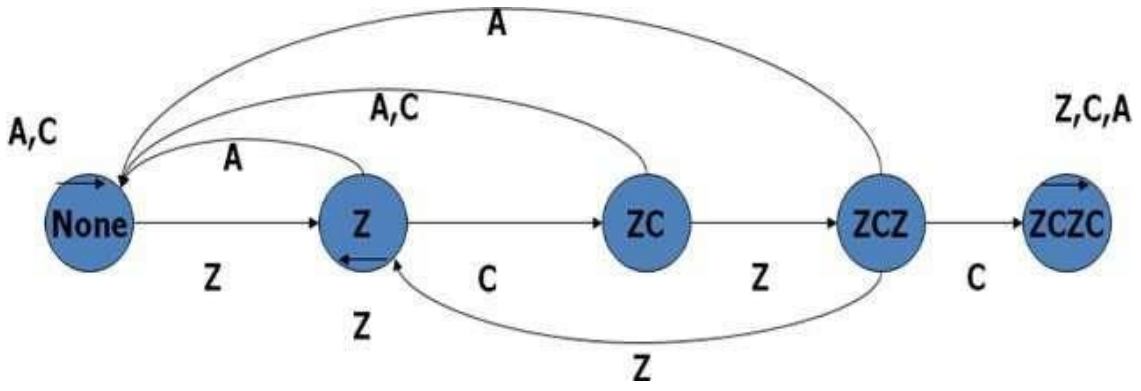
4. For example, a moving automobile whose engine is running can have the following states with respect to its transmission. Reverse gear , Neutral gear , First gear ,Second gear ß, Third gear , Fourth gear

State graph - Example

For example, a program that detects the character sequence "ZCZC" can be in the following states.

5. Neither ZCZC nor any part of it has been detected.

Z has been detected.
ZC has been detected.
ZCZ has been detected.
ZCZC has been detected.



States are represented by Nodes. State are numbered or may identified by words or whatever else is convenient.

Inputs and Transitions

6. Whatever is being modelled is subjected to inputs. As a result of those inputs, the state changes, or is said to have made a Transition.
7. Transitions are denoted by links that join the states.

8. The input that causes the transition are marked on the link; that is, the inputs are link weights.
9. There is one out link from every state for every input.

If several inputs in a state cause a transition to the same subsequent state, instead of drawing a bunch of parallel links we can abbreviate the notation by listing the several inputs as in: “input1, input2, input3……”.

Finite State Machine

10. A finite state machine is an abstract device that can be represented by a state graph having a finite number of states and a finite number of transitions between states. Outputs
11. An output can be associated with any link.

Out puts are denoted by letters or words and are separated from inputs by a slash as follows: “input/output”.

As always, output denotes anything of interest that’s observable and is not restricted to explicit outputs by devices.

1. Outputs are also link weights.
2. If every input associated with a transition causes the same output, then denoted it as:

“input1, input2, input3……/output”

State tables

1. 1. Big state graphs are cluttered and hard to follow.
2. 3. It’s more convenient to represent the state graph as a table (the state table or state transition table) that specifies the states, the inputs, the transitions and the outputs.
4. 5. The following conventions are used:
6. 7. Each row of the table corresponds to a state.
8. 9. Each column corresponds to an input condition.
10. 11. The box at the intersection of a row and a column specifies the next state (the transition) and the output, if any.

State Table-Example

inputs

Time Versus

STATE	Z	C	A
NONE	Z	NONE	NONE
Z	Z	ZC	NONE
ZC	ZCZ	NONE	NONE
ZCZ	Z	ZCZC	NONE
ZCZC	ZCZC	ZCZC	ZCZC

Sequence

State graphs don't represent time-they represent sequence.

A transition might take microseconds or centuries;

A system could be in one state for milliseconds and another for years- the state graph would be the same because it has no notion of time.

1. Although the finite state machines model can be elaborated to include notions of time in addition to sequence, such as time Petri Nets.

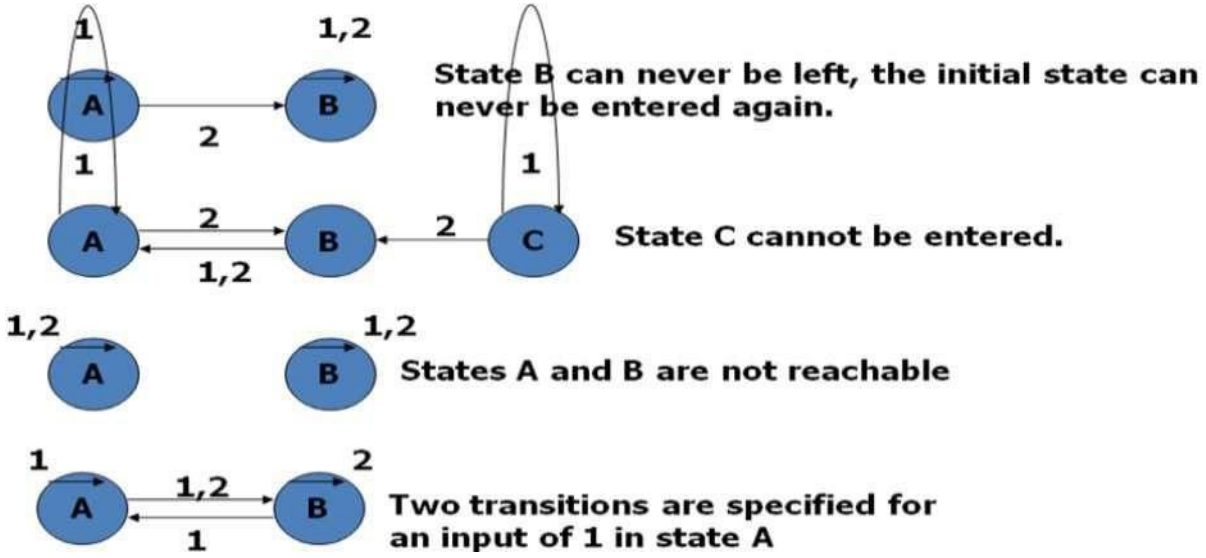
o Software implementation

2. There is rarely a direct correspondence between programs and the behavior of a process described as a state graph.
3. The state graph represents, the total behavior consisting of the transport, the software, the executive, the status returns, interrupts, and so on.
1. There is no simple correspondence between lines of code and states. The state table forms the basis.

Good and Bad State Graphs

2. What constitutes a good or a bad state graph is to some extent biased by the kinds of state graphs that are likely to be used in a software test design context.
3. Here are some principles for judging.
    1. The total number of states is equal to the product of the possibilities of factors that make up the state.
    2. For every state and input there is exactly one transition specified to exactly one, possibly the same, state.
    3. For every transition there is one output action specified. The output could be trivial, but at least one output does something sensible.
    4. For every state there is a sequence of inputs that will drive the system back to the same state.

Important graphs



State Bugs-Number of States

4. The number of states in a state graph is the number of states we choose to recognize or model.
5. The state is directly or indirectly recorded as a combination of values of variables that appear in the data base.
6. For example, the state could be composed of the value of a counter whose possible values ranged from 0 to 9, combined with the setting of two bit flags, leading to a total of 2*2*10=40 states.
7. The number of states can be computed as follows: o Identify all the component factors of the state. o Identify all the allowable values for each factor.

The number of states is the product of the number of allowable values of all the factors.

8. Before you do anything else, before you consider one test case, discuss the number of states you think there are with the number of states the programmer thinks there are.

There is no point in designing tests intended to check the system's behavior in various states if there's no agreement on how many states there are.

Impossible States

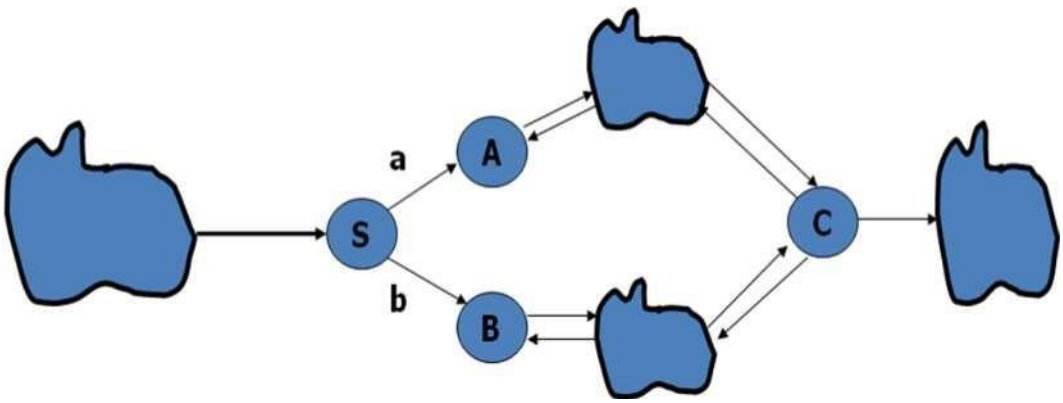
9. Some times some combinations of factors may appear to be impossible.

The discrepancy between the programmer's state count and the tester's state count is often due to a difference of opinion concerning "impossible states".

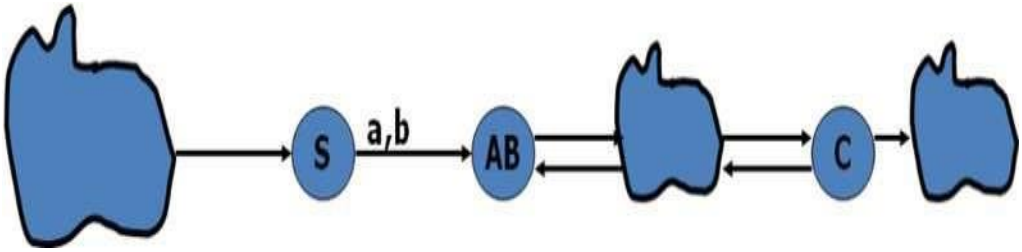
10. A robust piece of software will not ignore impossible states but will recognize them and invoke an illogical condition handler when they appear to have occurred.

Equivalent States

11. Two states are Equivalent if every sequence of inputs starting from one state produces exactly the same sequence of outputs when started from the other state. This notion can also be extended to set of states.



Merging of Equivalent States



Recognizing Equivalent States

Equivalent states can be recognized by the following procedures:
The rows corresponding to the two states are identical with respect to input/output/next state but the name of the next state could differ.

12. There are two sets of rows which, except for the state names, have identical state graphs with respect to transitions and outputs. The two sets can be merged.

Transition Bugs:

unspecified and contradictory Transitions

Every input-state combination must have a specified transition.

If the transition is impossible, then there must be a mechanism that prevents the input from occurring in that state.

13. Exactly one transition must be specified for every combination of input and state.
A program can't have contradictions or ambiguities.
14. Ambiguities are impossible because the program will do something for every input. Even the state does not change, by definition this is a transition to the same state.

Unreachable States

1. An unreachable state is like unreachable code.
2. A state that no input sequence can reach.
3. An unreachable state is not impossible, just as unreachable code is not impossible There may be transitions from unreachable state to other states; there usually because the state became unreachable as a result of incorrect transition.
4. There are two possibilities for unreachable states:
There is a bug; that is some transitions are missing.
The transitions are there, but you don't know about it.

Dead States

A dead state is a state that once entered cannot be left. This is not necessarily a bug but it is or suspicious.

OUT PUT ERROURS

1. The states, transitions, and the inputs could be correct, there could be no dead
2. unreachable states, but the output for the transition could be incorrect.
3. Output actions must be verified independently of states and transitions. State Testing
4. Impact of Bugs
5. If a routine is specified as a state graph that has been verified as correct in all details.
6. Program code or table or a combination of both must still be implemented.
7. A bug can manifest itself as one of the following symptoms:
8. Wrong number of states.
9. Wrong transitions for a given state-input combination.
10. Wrong output for a given transition.
11. Pairs of states or sets of states that are inadvertently made equivalent. States or set
12. of states that are split to create in equivalent duplicates.
13. States or sets of states that have become dead.
14. States or sets of states that have become unreachable.

Principles of State Testing

1. The strategy for state testing is analogous to that used for path testing flow graphs.

   Just as it's impractical to go through every possible path in a flow graph, it's impractical to go through every path in a state graph.

1. The notion of coverage is identical to that used for flow graphs.

Even though more state testing is done as a single case in a grand tour, it's impractical to do it that way for several reasons.

2. In the early phases of testing, you will never complete the grand tour because of bugs. Later, in maintenance, testing objectives are understood, and only a few of the states and transitions have to be tested. A grand tour is waste of time.
3. Theirs is no much history in a long test sequence and so much has happened that verification is difficult.

Starting point of state testing

4. Define a set of covering input sequences that get back to the initial state when starting from the initial state.
5. For each step in each input sequence, define the expected next state, the expected transition, and the expected output code.
6. A set of tests, then, consists of three sets of sequences: ○ Input sequences
   1. Corresponding transitions or next-state names ○ Output sequences

Limitations and Extensions

1. State transition coverage in a state graph model does not guarantee complete testing. How
2. defines a hierarchy of paths and methods for combining paths to produce covers of state graphs.
3. The simplest is called a "0 switch" which corresponds to testing each transition individually.
4. The next level consists of testing transitions sequences consisting of two transitions called "1 switches".
5. The maximum length switch is "n-1 switch" where there are n numbers of states.
6. Situations at which state testing is useful
7. Any processing where the output is based on the occurrence of one or more sequences of events, such as detection of specified input sequences, sequential format validation,
8. parsing, and other situations in which the order of inputs is important.
9. Most protocols between systems, between humans and machines, between components of a system.

Device drivers such as for tapes and discs that have complicated retry and recovery procedures if the action depends on the state.
Whenever a feature is directly and explicitly implemented as one or more state transition tables.

Testability Tips

1. **Design for Simplicity** – Keep the software simple and modular to make testing easier.
2. **Ensure Observability** – Make outputs, logs, and error messages easy to inspect.
3. **Improve Controllability** – Allow testers to easily control inputs and system states.
4. **Use Modular Design** – Divide the system into independent modules for unit testing.
5. **Provide Clear Interfaces** – Well-defined interfaces simplify testing and integration.

6. **Enable Automation** – Design the software so test cases can be automated.
7. **Maintain Good Documentation** – Requirements, design, and test documents should be clear and up to date.
8. **Include Error Logging** – Record detailed logs to help identify and fix defects.
9. **Use Assertions and Debugging Tools** – Detect faults early during development.
10. **Design for Reusability** – Reusable components and test cases reduce testing effort.